

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	V Adarsh Reddy	Reg.No.:	192525412
Department:	B Tech AI ML	Date:	/08/2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2' s complement representation. Consider two numbers: +45 and - 23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

A) Signed 8-bit Arithmetic Using 2's Complement (+45 and - 23)

Step 1: Convert Decimal to Binary

+45

Binary of 45 = 00101101

- 23

Binary of 23 = 00010111

Take 2's complement:

- 1's complement = 11101000
- Add 1 = 11101001

So,

- 23 = 11101001

A. Addition: $(+45) + (-23)$

00101101 (+45)
+ 11101001 (-23)

1 00010110

Ignore the carry out.

Result = 00010110

Binary:

00010110 = 22

Answer = +22

Overflow Check

Overflow occurs only when:

- Two positive numbers give a negative result.
- Two negative numbers give a positive result.

Here,

- One number is positive.
- One number is negative.

Overflow = No

B. Subtraction: $(+45) - (-23)$

Subtracting a negative means adding its positive.

$45 - (-23)$

= $45 + 23$ Binary:

+45 = 00101101

+23 = 00010111

Addition:

00101101
+ 00010111

01000100

Binary:

01000100 = 68

Answer = +68

Overflow Check Both

operands are positive.

Result is also positive.

Since 68 lies within the 8-bit signed range

- 128 to +127

Overflow = No

How Processor Detects Overflow

Processor compares:

- Carry into MSB
- Carry out of MSB

If they are different,

Overflow = 1

Otherwise,

Overflow = 0

Another rule:

- Positive + Positive = Negative → Overflow
- Negative + Negative = Positive → Overflow

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry lookahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

A) 4-bit Carry Look-Ahead Adder (CLA)

Suppose

A = 1011

B = 0110

Cin = 0

Step 1: Generate (G) and Propagate (P)

Formula

$G_i = A_i \times B_i$

$P_i = A_i \oplus B_i$

Bit	A	B	Generate (G)	Propagate (P)
0	1	0	0	1
1	1	1	1	0
2	0	1	0	1
3	1	0	0	1

Step 2: Carry Equations

$C_1 = G_0 + P_0C_{in}$

$C_2 = G_1 + P_1G_0 + P_1P_0C_{in}$

$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{in}$

$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + \dots$

Carries are computed simultaneously.

Ripple Carry Delay

Ripple Carry:

$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$

Each stage waits for the previous carry.

Delay

$\approx 4 \times \text{gate delay}$

CLA Delay

All carries are calculated in parallel.

Delay

$\approx 2 \text{ gate delays}$

Why CLA is Faster

- Carries generated simultaneously
- No waiting between stages
- Faster ALU performance
- Used in high-speed processors

Comparison

Feature	Ripple Carry	Carry Look-Ahead
Carry generation	Sequential	Parallel
Speed	Slow	Fast
Hardware	Simple	Complex
Delay	High	Low

3. In a signed multiplication operation, a processor uses Booth’s algorithm to multiply two numbers, - 6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

A) Booth's Algorithm (- 6 × +5)

Given

Multiplicand (M) = - 6

Multiplier (Q) = +5

Binary (4-bit)

M = 1010

- M = 0110

Q = 0101

A = 0000

$Q_{-1} = 0$

Initial

A = 0000

Q = 0101

$Q_{-1} = 0$

Cycle 1

$Q_0 Q_{-1}$

10

Subtract M

$A = A - M$

$0000 - 1010$

= 0110

Arithmetic shift

A=0011

Q=0010

Q- 1=1

Cycle 2

Q0Q- 1

01

Add M

0011+1010

=1101

Shift

A=1110

Q=1001

Q- 1=0

Cycle 3

10

Subtract M

1110- 1010

=0100

Shift

A=0010

Q=0100

Q- 1=1

Cycle 4

01

Add M

0010+1010

=1100

Shift

A=1110

Q=0010

Final Result AQ

11100010

Decimal

- 30

Since - 6

$\times 5 = - 30$

Correct answer.

Why Booth's Algorithm is Efficient

- Works directly with signed numbers.
- Consecutive 1s require only one addition and one subtraction.
- Fewer arithmetic operations.

Faster than ordinary shift-and-add multiplication

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

A) Division of $13 \div 3$

Dividend = 13 = 1101

Divisor = 3 = 0011

A. Restoring Division

Initial

A = 0000

Q = 1101

M = 0011

Perform 4 iterations:

Each iteration:

1. Left shift AQ
2. $A = A - M$
3. If $A < 0$

Restore

$A = A + M$

Q0 = 0

Otherwise

Q0 = 1

Final

Quotient = 0100 = 4

Remainder = 0001 = 1

B. Non-Restoring Division

Initial

A = 0000

Q = 1101

Each iteration

- Shift
- If A positive

$A = A - M$

- If A negative

$A = A + M$

No restoring after every subtraction.

Final correction performed only once.

Result

Quotient = 0100 = 4

Remainder = 0001 = 1

Comparison

Feature	Restoring	Non-Restoring
Restoration	Yes	No
Operations	More	Less
Feature	Restoring	Non-Restoring
Speed	Slower	Faster
Hardware Simple Efficient		

Why Non-Restoring is Preferred

- Eliminates unnecessary restoration.
- Reduces execution time.
- Better processor performance.
- Widely used in modern CPUs.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., - 13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

A)

IEEE 754 Floating-Point Representation

Given

- 13.25

Step 1: Convert to Binary

Integer

$13 = 1101$

Fraction

$0.25 = 0.01$

Therefore

$13.25 = 1101.01_2$

Normalize

1.10101×2^3

A. Single Precision (32-bit)

Sign

Negative

$S = 1$

Exponent

Bias

127

Exponent

$3 + 127 = 130$

Binary
10000010

Mantissa

Drop leading 1

101010000000000000000000

Final Representation

Sign		Mantissa
	Exponent	
1		10000010
101010000000000000000000		

B. Double Precision (64-bit)

Sign

1

Exponent

Bias

1023

$3+1023=1026$

Binary

1000000010

Mantissa

1010100

Final Representation

Field	Bits
Sign	1
Exponent	1000000010
Field	Bits
Mantissa 1010100	

Floating-Point Addition Steps

Suppose

$$13.25 + 5.5$$

Step 1

Convert both numbers to IEEE format.

Step 2

Compare exponents.

Step 3

Shift the mantissa of the smaller exponent until both exponents match.

Step 4

Add the mantissas.

Step 5

Normalize the result if necessary.

Step 6

Round according to the IEEE 754 rounding rule (typically Round to Nearest, Even).

Step 7

Store the sign, exponent, and mantissa.

Issues in Floating-Point Arithmetic

- Normalization: Ensures the result is in the form $1.x \ x \ x \ x \ x \times 2^n$.
- Rounding: If extra bits remain after the mantissa limit, they are rounded according to IEEE 754 rules.
- Precision Loss: Some decimal fractions (e.g., 0.1) cannot be represented exactly in binary, leading to small approximation errors.
- Overflow/Underflow: Very large results may overflow to infinity, while very small values may underflow to zero or become denormalized.

These IEEE 754 rules provide a consistent and standardized way of representing and computing real numbers across different computer systems.

Code of Conduct:

I V Adarsh Reddy (192525412) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

V ADARSH REDDY
Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation